

Потоковый ответ и отмена

Основной план, промпт 1.3

Краткое содержание

Потоковый ответ — это не постепенно растущий `ModelResponse`. Это упорядоченная последовательность предварительных событий `ModelEvent`, которая заканчивается одним подтверждённым исходом. Полученный фрагмент текста можно показать пользователю, но нельзя записывать как завершённый ход агента. Частично полученные аргументы инструмента нельзя исполнять. Отмена, истечение крайнего срока и обрыв соединения прекращают ожидание по разным причинам, поэтому среда выполнения обрабатывает их по-разному. Повтор всегда начинает новую попытку модели, расходует новый лимит и может привести к другому тексту или другому вызову инструмента.

Ключевые понятия

- **Наблюдённая часть потока** — точная последовательность уже полученных событий. Она полезна для интерфейса, аудита и диагностики, но ещё не является завершённым ответом.
- **Завершающее событие** — событие, после которого результат больше не меняется. В текущем общем контракте это `ResponseCompleted`.
- **Предварительное состояние** — обратимое представление незавершённого ответа, например текст на экране во время генерации. Это ещё не зафиксированное состояние агента.
- **Обрыв потока** — поток закончился без завершающего события из-за конца соединения, транспортного сбоя или ошибки чтения.
- **Отмена вызывающей стороной** — инициатор запроса больше не хочет продолжать операцию. В `asyncio` этот сигнал представлен исключением `CancelledError`.
- **Крайний срок** — абсолютный момент, после которого вся операция должна завершиться, а не новый локальный тайм-аут на каждом слое.
- **Обратное давление** — чтение ограничивается скоростью получателя, чтобы очередь событий и потребление памяти не росли без границ.
- **Лимит повторов** — общий предел числа попыток, времени и стоимости. Решение о повторе принадлежит всей операции, а не отдельному адаптеру.

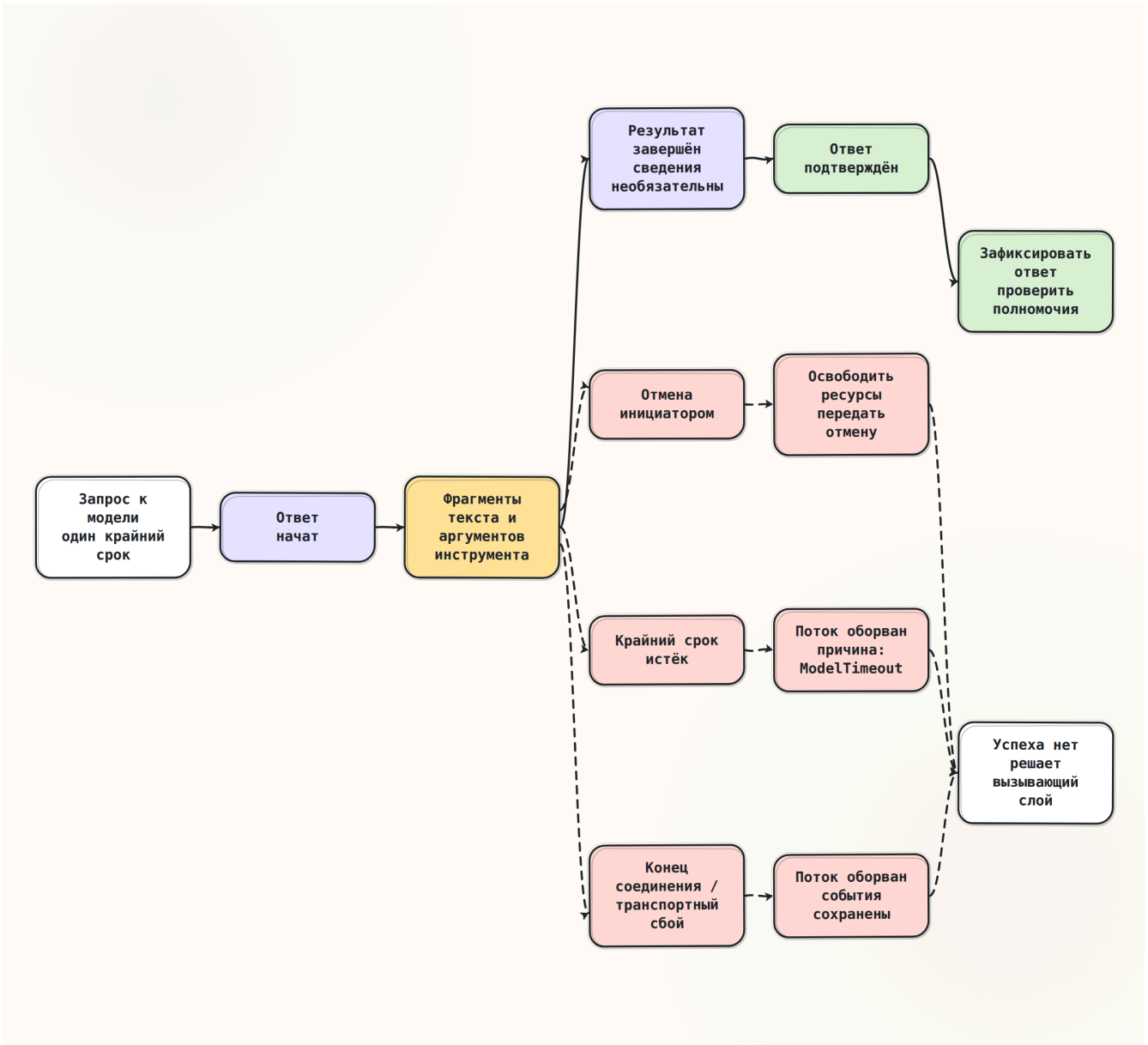
Основные идеи

Один поток, две шкалы состояния

Транспорт уже может передать десятки фрагментов, хотя смысловой ответ модели ещё не завершён. Поэтому среда выполнения хранит два разных представления:

1. `observed_events` пополняется после каждого проверенного события;
2. `committed_response` остаётся пустым до `ResponseCompleted`.

Это различие защищает агента от преждевременного решения. Видимый текст можно удалить или пометить незавершённым. Исполненный инструмент уже мог изменить внешнюю систему, поэтому `ToolArgumentsDelta` остаётся лишь частью строки до `OutputCompleted`, завершающего события и проверки полномочий.



Главный инвариант: ни конец соединения, ни отмена, ни истечение срока не создают искусственный `ResponseCompleted`. Только настоящее завершающее событие разрешает перенести ответ из предварительного состояния в зафиксированное состояние агента.

Жизненный цикл успешного ответа

1. Среда выполнения создаёт `ModelRequest`, идентификатор попытки и один абсолютный крайний срок.

2. Адаптер открывает поток; `ResponseStarted` связывает его с запросом и ответом поставщика.
3. `TextDelta` и `ToolArgumentsDelta` расширяют наблюдаемую часть потока. Порядок событий, поставщик и индекс результата проверяются сразу.
4. `OutputCompleted` подтверждает целостность одного результата. Для JSON, полученного частями, собранные фрагменты должны совпасть с полным исходным значением.
5. `UsageReported` может прийти поздно или отсутствовать.
6. `ResponseCompleted` сверяет запрос, поставщика, завершённые результаты и сведения об использовании. Затем `ModelResponse` проверяет допустимое сочетание исхода и вариантов результата. Только после этого ответ можно передать агенту.

`ResponseOutcome.TRUNCATED` тоже означает завершение потока: поставщик прекратил генерацию из-за лимита. Ответ модели неполон, но транспорт не сломан. Решение увеличить лимит токенов, запросить продолжение или выбрать другую модель создаёт отдельный запрос.

Отмена и крайний срок — разные причины

Оба механизма могут технически отменить ожидающую сопрограмму, но выражают разное намерение:

- внешний `CancelledError` означает, что инициатор больше не ждёт результата; среда выполнения прекращает чтение и отслеживание срока, закрывает источник и передаёт отмену выше без скрытого повтора;
- истечение срока до первого события либо во время ожидания перед повтором даёт `ModelTimeout` с указанием этапа;
- истечение срока после предварительных событий, но до проверенного `ResponseCompleted`, даёт `ModelStreamInterrupted` с полученными событиями и причиной `ModelTimeout`, а не завершённый ответ.

Крайний срок передаётся как абсолютное значение по монотонным часам. Иначе подключение, чтение потока и каждый повтор получают полный интервал заново, а вся операция превышает срок, обещанный пользователю.

В Python 3.13 `asyncio.timeout_at(deadline)` принимает абсолютное время цикла событий. Контекст отменяет работу внутри и преобразует только собственное истечение в `TimeoutError` снаружи. До подтверждения ответа обычный `CancelledError` нельзя поглощать: иначе нарушаются освобождение ресурсов и структурированная конкурентность. После проверенного `ResponseCompleted` текущая политика делает узкое исключение: завершает освобождение ресурсов, возвращает уже подтверждённый ответ и записывает проблему в `cleanup_error`.

Ответ 429 и заголовок `Retry-After`

Код HTTP 429 Too Many Requests сообщает об ограничении частоты запросов. Заголовок `Retry-After` подсказывает длительность ожидания, но не гарантирует успех и не разрешает превышать общий лимит времени.

Среда выполнения может начать новую попытку, только если одновременно:

- политика разрешает повтор на текущем этапе;
- число попыток и ожидаемая стоимость укладываются в общий лимит;
- ожидание и новая попытка помещаются в оставшееся время;
- ожидание можно прервать отменой инициатора;
- предыдущая попытка не изменила состояние агента и не создала внешний побочный эффект.

Адаптер возвращает типизированную ошибку `ModelRateLimited(retry_after_s=...)`, а решение об ожидании принимает вызывающий слой. Если корректного `Retry-After` нет, промышленная среда обычно применяет ограниченную экспоненциальную задержку со случайным разбросом. В детерминированной практике используется постоянная резервная задержка. Если она не помещается в оставшееся время, результатом становится тайм-аут.

Когда повтор допустим

«Безопасный повтор» здесь означает прежде всего отсутствие повторного побочного эффекта. Даже чистая генерация с тем же запросом может дать другой ответ, другую стоимость и другую задержку.

Наблюдение	Решение среды выполнения
429 до первого события	Повтор допустим после <code>Retry-After</code> , если хватает времени и общего лимита попыток
Срок истёк до первого события	Внутри этой операции повтор невозможен: время исчерпано. Нужен новый запрос с новым сроком
Обрыв после фрагмента текста	Сохранить фрагмент как незавершённый; новую попытку не дописывать к старой
Обрыв внутри аргументов инструмента	Не разбирать и не исполнять частичный JSON; новая попытка формирует предложение заново
Отмена инициатором	Освободить ресурсы и передать <code>CancelledError</code> выше; автоматический повтор запрещён
Завершение с <code>TRUNCATED</code>	Это не транспортный повтор; требуется отдельное решение о продолжении, другой модели или остановке
<code>usage=None</code>	Не повторять; стоимость остаётся неизвестной, а не нулевой

Если незавершённый текст уже показан пользователю, новая попытка должна заменить его или начать отдельный ответ с явной пометкой. Склейка старого фрагмента и новой генерации создаёт текст, которого целиком не выдавала ни одна попытка модели.

Причинная связь с агентом

- `TextDelta` → меняется только обратимое представление в интерфейсе → инструмент недоступен → обрыв не создаёт внешнего эффекта.

- `OutputCompleted` → один результат целостен, но весь ответ ещё не подтверждён → среда выполнения продолжает ждать → параллельные результаты и сведения об использовании не теряются.
- `ResponseCompleted` → появляется подтверждённый `ModelResponse` → агент может принять следующее решение. Предложение инструмента отдельно проходит проверку схемы, списка разрешённых инструментов и полномочий до исполнения.
- Отмена, истечение срока или обрыв до проверенного `ResponseCompleted` → подтверждённого ответа нет → общая политика решает, допустима ли новая попытка → адаптер не скрывает новое решение модели.

Примеры

Предварительный текст без преждевременной фиксации

```
async def preview(source):
    async for event in source:
        audit.append(event)
        if isinstance(event, TextDelta):
            ui.preview(event.delta) # Обратимое представление, не состояние агента
    yield event

collected = await collect_stream(preview(client.stream(request)))
state.commit(collected.response) # Только после проверки жизненного цикла
```

Если после `TextDelta("Возьмите зон")` соединение оборвалось, интерфейс может показать незавершённый фрагмент ответа. Но этот текст не добавляется в историю агента как обычное сообщение модели. Если поставщик завершил ответ с `outcome=TRUNCATED` и `usage=None`, состояние честно хранит оба факта: ответ неполон, а стоимость неизвестна.

Один абсолютный крайний срок

В практике эту границу реализует `StreamingRuntime`: один объект `Deadline` передаётся каждой видимой попытке, ограничивает чтение потока и ожидание перед повтором.

```
deadline = Deadline(clock.now() + 10)
result = await StreamingRuntime(time=clock).collect(
    client,
    request,
    deadline=deadline,
)
```

`Deadline` относится к одному источнику монотонного времени. Его нельзя передавать между процессами как обычную временную метку. После перезапуска нужно сохранить переносимый момент настенных часов или оставшееся время, а затем построить новый локальный крайний срок по монотонным часам.

Заметки и наблюдения

Что доказано практикой 1.3

- `StreamingRuntime` создаёт `ModelAttempt`, передаёт один абсолютный `Deadline` каждой попытке и ограничивает им чтение потока и ожидание перед повтором.
- Отмена инициатором доходит до заблокированного источника как `CancelledError`: источник закрывается, задача слежения за сроком удаляется, новая попытка не открывается.
- Транспортный сбой или истечение срока после принятых событий возвращаются как `ModelStreamInterrupted` с точными объектами событий и исходной причиной. Ни `ModelResponse`, ни искусственное завершающее событие не создаются.
- `ModelRateLimited` до первого события допускает ограниченный повтор. После первого события та же ошибка становится причиной `ModelStreamInterrupted`, поэтому среда выполнения не склеивает две генерации.
- `ResponseOutcome.TRUNCATED` остаётся подтверждённым завершением, а отсутствующие сведения об использовании сохраняются как `None`.
- После проверенного `ResponseCompleted` ошибка или отмена во время освобождения ресурсов записывается в `cleanup_error` и не отменяет подтверждённый ответ.

Две наблюдаемые трассы находятся в `demo.py`, а матрица отказов — в `test_execution.py`.

Итог инженерной защиты: полномочия инструмента

В сценарии после синтаксически полного `ToolArgumentsDelta({'sku': 'sku-42', 'quantity': 2})` транспорт обрывается до `OutputCompleted` и `ResponseCompleted`.
Выбранная политика:

1. Агент остаётся в состоянии ожидания подтверждённого решения; полученная часть потока — черновик, а не `ToolCallOutput`.
2. Среда выполнения возвращает `ModelStreamInterrupted` без скрытого повтора.
3. Приложение может предложить отмену или отдельную новую генерацию после решения пользователя. Ручное подтверждение не означает исполнение JSON из оборванного потока: отдельная команда снова проходит проверку схемы и полномочий.
4. Только `ToolCallOutput` из одного проверенного `ResponseCompleted` может стать предложением вызвать `reserve_inventory`. Затем обязательны проверка схемы, списка разрешённых инструментов и полномочий. События разных попыток не объединяются.

Проверяющий тест

`test_complete_tool_arguments_interruption_never_retries_or_authORIZES_tool` содержит отрицательную и положительную ветви. В ветви с обрывом полный JSON остаётся событием, попытка остаётся одна, а тестовый инструмент не вызывается. В отдельном новом запросе последовательность `OutputCompleted` → `ResponseCompleted` создаёт допустимое предложение, после чего тестовая политика разрешает ровно один вызов. Эксперимент доказывает границу завершения, но не заменяет полноценную промышленную проверку полномочий.

Ограничения текущего доказательства

- `ControlledStreamingClient` и `ManualTime` доказывают локальный порядок выполнения, но не удалённую отмену, остановку вычислений поставщика или прекращение тарификации после закрытия соединения.
- Здесь нет настоящего адаптера HTTP/SSE, долговечного сохранения состояния и воспроизведения после падения процесса, а также отдельного лимита стоимости.
- Резервная задержка детерминирована и не моделирует случайный разброс или конкуренцию нескольких рабочих процессов.
- `collect_stream` сохраняет все события и фрагменты в памяти; ограничение буфера и сквозное обратное давление не измерены.
- При отмене инициатором исключение передаётся наружу без полученных событий. Для аудита события нужно записывать в отдельное хранилище до подтверждения ответа.
- Общего протокола продолжения нет. Новый запрос начинает отдельную попытку модели, а идентификатор связи сам по себе не обеспечивает идемпотентность.

Гонки на границе завершения

Тесты фиксируют обе стороны границы. Если крайний срок и ещё не принятое завершающее событие готовы на одном обороте цикла событий, побеждает крайний срок и завершённый ответ не создаётся. Если `ResponseCompleted` уже проверен, а срок истекает или приходит отмена во время освобождения ресурсов, побеждает подтверждённый ответ; проблема освобождения остаётся наблюдаемой через `cleanup_error`.

Вопросы для повторения

1. Почему полученный фрагмент текста ещё не является `ModelResponse`?
2. Как частичный `ToolArgumentsDelta` может превратить транспортную ошибку в опасный побочный эффект?
3. Почему отмену инициатором нельзя автоматически преобразовать в тайм-аут?
4. Зачем передавать один абсолютный крайний срок через транспорт и ожидание перед повтором?
5. Когда ответ `429` допускает повтор и почему одного `Retry-After` мало?
6. Почему `TRUNCATED` означает завершение ответа, а не транспортный сбой?
7. Что нужно сохранить, если сведения об использовании отсутствуют?
8. Почему новую генерацию нельзя дописать к последнему фрагменту старой без явного протокола продолжения?

Итоги

Потоковая выдача сокращает время до первого видимого фрагмента, но усложняет границу фиксации. Наблюдённая часть потока относится к предварительному состоянию; только проверенное завершающее событие создаёт `ModelResponse`. Отмена сохраняет намерение инициатора, крайний срок ограничивает всю операцию, а обрыв сохраняет точную незавершённую часть потока. Повтор остаётся явным решением среды выполнения с общими лимитами времени и числа попыток и никогда не маскируется под продолжение прежнего

ответа. Отдельный лимит стоимости в практике не реализован и остаётся обязанностью вызывающего слоя.

Практика завершена без фреймворка: реализованы потоковая среда выполнения с единым крайним сроком, детерминированные часы и сценарии отказов для обрыва, отмены, ответа 429, лимита токенов, отсутствующих сведений об использовании и освобождения ресурсов после завершения. Карта запуска и ограничения сборки в README.

Источники

Изменчивые особенности Python и HTTP проверены по первичным источникам 2026-08-02; локальная практика повторно запущена 2026-08-12 на Python 3.13.13:

- Python 3.13: отмена задач
- Python 3.13: тайм-ауты и `asyncio.timeout_at`
- RFC 6585: ответ 429 Too Many Requests
- RFC 9110: заголовок `Retry-After`
- Основной план: промпт 1.3
- Локальные материалы: контракты, сборка потока, среда выполнения, тесты, README и конспект 1.2